UNIVERSITI TEKNOLOGI MARA
MARKING SCHEME

COURSE	:	COMPUTER ORGANIZATION AND ARCHITECTURE
COURSE CODE	:	CSC429
EXAMINATION	:	NOVEMBER 2026 (MOCK)
TIME	:	3 HOURS

MARKING SCHEME NOTES

1. This marking scheme accompanies the CSC429 mock paper (November 2026). Generated answers show working. Real-bank questions carry a labelled unofficial AI model answer, not an official scheme.

2. Marking points show the answer and mark allocation. Check model-generated real-bank points against the source paper.

3. Part A = 4 x 5 + 6 x 10; Part B = 1 x 20; total = 100 marks.

4. Generated slots: 2. Real-bank slots: 9. Unofficial model answers:

   9/9.
   For local study use only; model answers are unofficial.

5. Structure from 20 unique CSC429 past papers; every real question remains traceable to its source page.

FOR MARKERS' USE ONLY

This marking scheme consists of 9 printed pages

© Hak Cipta Universiti Teknologi MARA

PART A — MARKING GUIDE

QUESTION A1 (5 marks)

Topic: Parallel and multiprocessor architecture

Source: CSC429 (5).pdf, page(s) 6

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

The question asks for completion of a taxonomy of parallel processor architecture with appropriate examples. The standard taxonomy (Flynn's classification) divides parallel processors by the number of instruction streams and data streams: SISD, SIMD, MISD and MIMD. A completed taxonomy with examples is: SISD — a single uniprocessor (single control unit, single ALU, one instruction stream operating on one data stream), e.g. a conventional single-core PC processor. SIMD — one instruction stream operating on many data streams, e.g. an array processor or a GPU/vector processor where the same operation is applied to a vector of data. MISD — multiple instruction streams on a single data stream; no practical commercial example (sometimes cited as a systolic array or fault-tolerant replicated system), so it is largely theoretical. MIMD — multiple instruction streams on multiple data streams, e.g. a multicore processor or a cluster of workstations, where each processor executes its own program on its own data. If the figure instead uses the structural taxonomy (single processor vs. multiprocessor, and shared-memory/multiprocessor vs. distributed/multicomputer), the corresponding completion is: single processor (uniprocessor) as the non-parallel case; multiprocessor with shared memory (e.g. SMP multicore) and multicomputer with distributed memory (e.g. cluster/network of workstations) as the parallel cases. In either form, each leaf of the taxonomy must be labelled with a correct category name and a matching example.

Unofficial marking guide:

• 1 mark: correct top-level division of the taxonomy (e.g. by instruction/data streams, or single vs. multiple processors)

• 2 marks: correct naming of the parallel categories (e.g. SIMD and MIMD, or shared-memory multiprocessor and distributed-memory multicomputer)

• 2 marks: an appropriate, correct example given for each category filled in

Model confidence: medium.

Model note: The diagram is reproduced in the question paper. This text-only answer generator was not given the image, so the answer gives both the standard Flynn taxonomy and the structural taxonomy; match the completion to the branches actually shown.

QUESTION A2 (5 marks)

Topic: Computer generations and history

Source: CSC429 (6).pdf, page(s) 2

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Machine X is the Jacquard Loom (Jacquard weaving machine), developed by Joseph Marie Jacquard.

(b) The Jacquard Loom was developed in the First Generation era context — more precisely, it belongs to the pre-computer/mechanical era (early 1800s), and in the standard CSC429 generation scheme it is associated with the First Generation (mechanical/electromechanical foundations of computing, c. 1800s–1940s). It is the punched-card control concept that later influenced First Generation computers.

(c) The deck of punched cards functions like a program/instruction store — the closest computer component is memory (main memory / program storage, e.g. the stored-program memory or ROM holding the control instructions). Equivalently, it acts as the control unit's instruction source; the punched card deck = program stored in memory.

(d) Input: the deck of punched cards (the punched-card pattern/instructions) fed into the loom. Output: the woven fabric with the intricate woven pattern produced by the loom.

Unofficial marking guide:

• 1 mark: (a) correctly names the Jacquard Loom.

• 1 mark: (b) states the correct generation (First Generation / mechanical era as per course scheme).

• 1 mark: (c) names a valid analogous component (memory/program store or control unit instruction source).

• 2 marks: (d) 1 mark for input (punched card deck), 1 mark for output (woven patterned fabric).

Model confidence: medium.

© Hak Cipta Universiti Teknologi MARA

Model note: The placard text and figure are only partially available; 'Machine X' is inferred as the Jacquard Loom from the description of Joseph Marie Jacquard and punched-card-controlled weaving. Generation classification may vary by lecturer's scheme; the loom predates electronic computers and is usually treated as a precursor to First Generation punched-card machines.

QUESTION A3 (5 marks)

Topic: Data representation and number systems

Answer: left -53 -> 1 0110101; right 31 -> 0 0011111; -53 - 31 = -84 -> sign-magnitude 1 1010100

- *1 mark for converting both operands to sign-magnitude.*
- *1 mark for applying the sign rule for subtraction.*
- *1 mark for the correct magnitude operation.*
- *1 mark for the correct sign bit.*
- *1 mark for the final decimal and byte representation.*

QUESTION A4 (5 marks)

Topic: Instruction formats and addressing modes

Source: CSC429 (1).pdf, page(s) 5

Unofficial model answer — AI-generated and reviewer-corrected; not an official UTM marking scheme.

(a) The technique is direct addressing (also called absolute addressing). The instruction's address field A contains the effective address of the operand; the memory word at address A holds the operand itself (Address A -> Memory[A] = Operand).

(b) The CPU places Address A on the address bus and performs one memory read. The memory returns the operand from location A; the operand is loaded into the accumulator/register and the instruction executes. There is no second pointer fetch.

(c) Advantage: direct addressing is simple and needs only one memory access for the operand, so it is faster than indirect addressing and needs no extra pointer word. Disadvantage: the address field is fixed and limited in width, so it can address only a restricted range of memory and is less flexible for pointers, relocation or large address spaces.

Unofficial marking guide:

- *1 mark: correctly labels the technique as direct (absolute) addressing.*
- *1 mark: states that the address field A is the effective address of the operand.*
- *1 mark: explains the single memory access that fetches the operand from memory[A].*
- *1 mark: gives ONE valid advantage (simple; one memory access; no pointer word).*
- *1 mark: gives ONE valid disadvantage (limited address range; less flexible).*

Model confidence: high.

Model note: Reviewed against Figure 2: the diagram has one Memory box holding the Operand, so the address field identifies the operand directly; it is not indirect addressing.

QUESTION A5 (10 marks)

Topic: Input/output and interrupts

Source: CSC429 (16).pdf, page(s) 4

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Two methods to handle multiple interrupts: (1) Software polling / priority polling: When multiple interrupts occur, the processor branches to a common interrupt service routine that polls each device in priority order; the highest-priority requesting device is serviced first. This is flexible because priority can be changed in software, but it is slower because polling takes time. (2) Daisy chaining (hardware priority): Interrupt request lines are connected in a chain; the interrupt acknowledge signal propagates through devices in a fixed hardware priority order. The first device in the chain that has requested an interrupt is serviced first. This is fast and requires little software overhead, but priority is fixed by hardware and a faulty device can break the chain. (Other acceptable methods: vectored interrupts, multiple interrupt lines, or priority encoder.)

(b) One example of a non-maskable interrupt (NMI): a power failure or power-supply voltage drop (also acceptable: memory parity error, bus error, or a hardware reset button). Such an interrupt cannot be disabled by software and is used for critical events.

(c) DMA is faster than interrupt-driven and programmed I/O because the DMA controller transfers data directly between the I/O device and memory without the CPU executing instructions for each byte/word.

© Hak Cipta Universiti Teknologi MARA

The CPU only initializes the DMA (sets address, count, direction) and is interrupted once at the end of the block transfer. In programmed I/O the CPU continuously polls and moves each data item, and in interrupt-driven I/O the CPU is interrupted for every data item and executes an ISR, so both incur high CPU overhead and many bus cycles. DMA therefore achieves higher throughput and frees the CPU for other work.

(d) A 'hot connection' means the USB device can be connected or disconnected while the computer is powered on and running, without shutting down or rebooting the system. The interface is designed to be hot-swappable and supports plug-and-play detection and configuration of the device.

Unofficial marking guide:

• 1 mark: correctly naming the first method to handle multiple interrupts (e.g., software polling or daisy chaining).
• 1 mark: explaining how the first method works and its priority behaviour.
• 1 mark: correctly naming the second method to handle multiple interrupts.
• 1 mark: explaining how the second method works and its priority behaviour.
• 2 marks: giving one correct example of a non-maskable interrupt (1 mark for a valid example, 1 mark for indicating it is non-maskable/critical).
• 2 marks: explaining why DMA is faster (1 mark for direct memory-to-device transfer without CPU per-item involvement; 1 mark for contrast with CPU overhead in programmed/interrupt-driven I/O).
• 2 marks: defining hot connection (1 mark for connect/disconnect while powered on; 1 mark for no reboot/shutdown required or hot-swappable/plug-and-play).
Model confidence: high.
Model note: The question text contains '1/0' which is assumed to be 'I/O'. The answer lists two common methods; other valid methods (vectored interrupts, multiple interrupt lines) would also be acceptable. The marking guide allocates the full 10 marks as specified in the subparts.

QUESTION A6 (10 marks)

Topic: Memory hierarchy and cache

Source: CSC429 (8).pdf, page(s) 4, 5

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Ranking by price, cheapest to most expensive (per bit): tape back-up → hard drive → cache → register. Rationale: price per bit rises as memory becomes faster and closer to the CPU. Tape backup is the cheapest, slowest, highest-capacity offline medium; the hard drive (magnetic disk) is next, cheaper per bit than semiconductor memory; cache (SRAM, small and fast, on/near the CPU) is more expensive per bit than main memory/disk; registers are the fastest, smallest storage inside the CPU and hence the most expensive per bit.

(b) One benefit of cache memory: it reduces the average time to access memory (average memory access time), because frequently used instructions and data are held in fast SRAM close to the CPU. This exploits locality of reference (temporal and spatial), so most accesses hit in cache and avoid the much slower main memory, improving overall system performance/throughput and reducing the processor's memory stall time.

(c) Write through vs write back: In write through, every write by the CPU is written to both the cache block and the corresponding location in main memory simultaneously, so main memory is always kept up to date (consistent). It is simple but generates more main-memory traffic and is slower per write. In write back, the write is made only to the cache block; the modified block is marked dirty and is written back to main memory only when that block is replaced (evicted). This reduces main-memory writes and is faster/more efficient, but main memory can be temporarily out of date and the cache needs a dirty bit plus replacement-time write-back logic.

Unofficial marking guide:

• 1 mark: correct cheapest item (tape back-up) placed first
• 1 mark: hard drive placed second
• 1 mark: cache placed third
• 1 mark: register placed last (most expensive)
• 1 mark: states cache reduces average memory access time / speeds up memory access
• 1 mark: links benefit to locality of reference or avoiding slow main memory (explanation)
• 1 mark: write through writes to both cache and main memory on every write (memory always consistent)
• 1 mark: write back writes only to cache and updates main memory later on block replacement (dirty bit)
• 1 mark: correct comparison of traffic/performance (write through more memory traffic/slower; write back less traffic/faster)

© Hak Cipta Universiti Teknologi MARA

• 1 mark: correct comparison of consistency/complexity (write through simpler/consistent; write back needs dirty bit and can be temporarily inconsistent)

Model confidence: high.

Model note: Assumes 'price' means price per bit, the standard interpretation for memory-hierarchy ranking. The question text lists subparts as (a),(b),(c) but the slot label says A6; treated as one 10-mark question with 4+2+4 marks.

QUESTION A7 (10 marks)

Topic: Computer architecture and CPU organisation

Source: CSC429 (21).pdf, page(s) 3

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) The machine cycle is the sequence of steps the CPU repeats to fetch, decode, execute and (where needed) store results for each instruction. It is driven by the control unit and synchronised by the system clock. The stages and responsible components are:

1. Fetch: The program counter (PC) holds the address of the next instruction. The control unit copies this address to the memory address register (MAR) and asserts a memory-read signal on the control bus. Main memory returns the instruction word on the data bus into the memory data register (MDR), which is then transferred to the instruction register (IR). The PC is incremented to point to the next instruction. Responsible components: PC, MAR, MDR, main memory, control unit, buses.

2. Decode: The control unit examines the opcode in the IR and generates the appropriate control signals (using its decoder/sequencer). It determines the operation, the operands and the addressing mode. Responsible components: instruction decoder and control unit.

3. Execute: The arithmetic logic unit (ALU) performs the required arithmetic or logic operation on operands taken from registers or from memory (via MAR/MDR). The status/flag register records conditions such as zero, carry, sign and overflow. For a memory operand, the effective address is computed and a memory read or write is issued. Responsible components: ALU, general-purpose registers, flag register, control unit, MAR/MDR and memory for memory operands.

4. Store/write-back: The result is written back to a register or to main memory (via MDR and MAR with a memory-write signal), and the flags are updated. The cycle then repeats from fetch, unless a halt or interrupt occurs.

In short, the control unit coordinates the cycle, the PC/IR/MAR/MDR and buses move addresses and data, the ALU and registers perform and hold the computation, and main memory supplies instructions and operands.

(b) If main memory is corrupted, the fetch stage may read an invalid or altered instruction word into the IR, so the decoder produces wrong control signals and the CPU executes incorrect or undefined operations. Corrupted operands or data also give wrong results, and a corrupted address or parity/ECC error can cause a memory fault or exception. The control unit may then enter an error-handling routine, halt the machine, or (if no error detection is present) silently produce incorrect output. In short, the machine cycle cannot be trusted: fetch, decode and execute all operate on bad data, leading to wrong results, exceptions or a system crash.

Unofficial marking guide:

• 1 mark: identifies the four stages of the machine cycle (fetch, decode, execute, store/write-back) and that it repeats under control of the control unit and clock

• 2 marks: describes the fetch stage and names the responsible components (PC, MAR, MDR, IR, memory, buses, control unit)

• 2 marks: describes the decode stage and names the control unit/instruction decoder

• 2 marks: describes the execute stage and names the ALU, registers and flag/status register, including memory read/write for memory operands

• 1 mark: describes the store/write-back stage and the repeat of the cycle

• 1 mark: explains that corrupted memory causes wrong instructions/operands to be fetched, leading to incorrect execution, exceptions or a crash

• 1 mark: links the corruption to the responsible components (IR/decoder/control unit) and the consequence for the cycle

Model confidence: high.

Model note: The question text is complete; no figure or table is required. Marks are allocated as 8 for part (a) and 2 for part (b), totalling 10.

© Hak Cipta Universiti Teknologi MARA

QUESTION A8 **(10 marks)**

Topic: Input/output and interrupts

Source: CSC429 (2).pdf, page(s) 4

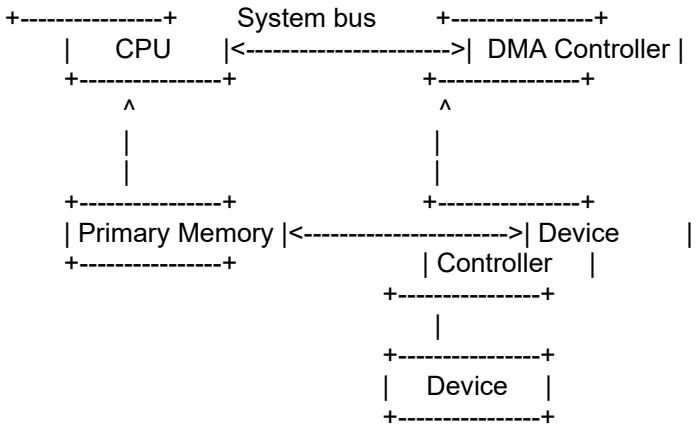
Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) DMA (Direct Memory Access) is an I/O technique in which a dedicated DMA controller transfers blocks of data directly between an I/O device (via its device controller) and primary memory without the CPU executing a load/store instruction for each data word. The CPU programs the DMA controller with the starting memory address, the device address, the transfer count and the direction (read or write); the DMA controller then performs the block transfer and interrupts the CPU only when the whole block is complete.

(b) DMA improves overall computer performance because the CPU is freed from supervising each individual word transfer. While the DMA controller moves data over the memory bus, the CPU can continue executing other instructions (or be idle), so the large overhead of per-word programmed or interrupt-driven transfers is removed. This increases effective CPU utilisation and system throughput, especially for high-volume I/O such as disk or network block transfers, and reduces the number of interrupts the CPU must service.

(c) Two major drawbacks of programmed and interrupt-driven I/O that motivated DMA:
1. CPU overhead per word: In programmed I/O the CPU repeatedly polls the device status and then executes an explicit load/store for every data word; in interrupt-driven I/O the CPU takes an interrupt and runs a service routine for every word (or small unit). This consumes many CPU cycles per word, so the CPU is tied up in I/O supervision and cannot do useful computation.
2. Poor throughput for block transfers: Because each word requires CPU intervention (polling loop or interrupt entry/exit), the achievable data rate is limited by CPU speed and interrupt latency, not by the device. For large blocks the CPU spends most of its time on I/O overhead, making these techniques inefficient for high-speed devices and bulk data movement.

(d) Redrawn DMA architecture (replacing Figure 2):



In the DMA architecture the DMA controller is an additional bus master placed between the device controller and primary memory (and connected to the CPU/system bus). Data flows directly between the device controller and primary memory under DMA control, while the CPU is connected to the same bus and can be granted or denied bus access by the DMA controller (cycle stealing or burst mode). The CPU no longer sits in the data path for each word; it only initialises the DMA controller and receives a completion interrupt.

Unofficial marking guide:

- *1 mark: DMA is a direct transfer between I/O device/controller and memory without CPU per-word involvement.*
- *1 mark: DMA controller is programmed by CPU with address/count/direction and interrupts CPU on completion.*
- *1 mark: CPU is freed from supervising each word transfer and can do other work during the block transfer.*
- *1 mark: Increased CPU utilisation/throughput and reduced interrupt overhead, especially for block/high-speed I/O.*
- *2 marks: First drawback clearly explained (CPU overhead per word in programmed/interrupt-driven I/O — polling or interrupt service per word).*
- *2 marks: Second drawback clearly explained (limited throughput / CPU tied up for block transfers / interrupt latency limits data rate).*
- *1 mark: DMA controller shown as an additional unit connected to the system bus between device controller and*

© Hak Cipta Universiti Teknologi MARA

memory.

• 1 mark: Data path shown directly between device controller and primary memory under DMA control, with CPU outside the per-word data path (CPU connected to bus and only initialises/receives interrupt).

Model confidence: high.

Model note: Figure 2 extracted text lists Primary Memory, CPU, Controller and Device; the redrawn diagram assumes a standard single-system-bus DMA arrangement with a DMA controller added as a bus master. The exact bus topology in the original figure should be checked against the paper figure if available.

QUESTION A9 (10 marks)

Topic: Computer architecture and CPU organisation

Source: CSC429 (15).pdf, page(s) 3

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Two bus types available in a computer system: (1) the data bus, which carries the actual data/operands between the CPU, memory and I/O; and (2) the address bus, which carries the memory or I/O port address that selects the source/destination. (Other acceptable answers: control bus, system bus, internal/CPU bus, expansion bus such as PCI.)

(b) System interconnection vs CPU interconnection — two differentiating characteristics:
1. Scope/function: System interconnection (the system bus) links the major components of the whole computer — CPU, main memory and I/O controllers — so it carries addresses, data and control signals between them. CPU interconnection is internal to the processor and links the CPU's own functional units (registers, ALU, control unit, cache), so it carries micro-architecture-level signals.
2. Speed/width and location: CPU interconnection is short, on-chip, and much faster/wider (higher bandwidth, matched to the processor clock), whereas system interconnection is longer (across the motherboard), shared by several devices, and slower/narrower, often requiring arbitration and buffering.

(c) Data transfer timing — the two main types:
- Synchronous timing: all bus events are referenced to a common clock. The address and data are placed on the bus at defined clock edges, and the transfer completes in a fixed, predetermined number of clock cycles. It is simple and fast for devices of similar speed, but the bus must run at the speed of the slowest device, so slow devices waste bus cycles.
- Asynchronous timing: there is no common clock; instead a handshake (e.g. request/acknowledge, or strobe/acknowledge) coordinates the transfer. The master issues a request/strobe and the slave returns an acknowledge when ready, so each device works at its own speed. It is flexible and accommodates mixed-speed devices but needs extra control lines and handshake overhead, making it slower per transfer.
(Some texts also mention a semi-synchronous compromise, which uses a clock but allows a wait/handshake for slow devices.)

Unofficial marking guide:

• 1 mark: correctly naming the data bus (with its function)
• 1 mark: correctly naming the address bus or another valid bus type (control/system bus) with its function
• 1 mark: first valid difference between system and CPU interconnection (scope/function)
• 1 mark: explanation/elaboration of that first difference
• 1 mark: second valid difference (speed/width/location)
• 1 mark: explanation/elaboration of that second difference
• 2 marks: synchronous timing explained (common clock, fixed cycles) with a merit/limitation
• 2 marks: asynchronous timing explained (handshake, no common clock) with a merit/limitation

Model confidence: high.

Model note: Question text has an OCR/line-break artefact ('system interconnection and CPU interconnection'); interpreted as comparing system-bus interconnection with CPU-internal interconnection. Part (c) answered with the two standard timing schemes (synchronous and asynchronous); semi-synchronous noted as an acceptable extra. This is an unofficial model answer, not an official marking scheme.

QUESTION A10 (10 marks)

Topic: Data representation and number systems

a) Answer: -31.510 = 0xC1FC0000 (sign=1, exponent=10000011, fraction=11111000000000000000000)

• 1 mark for sign, exponent and mantissa.
• 1 mark for the biased exponent.
• 1 mark for the 23-bit fraction.

- *1 mark for the 32-bit binary grouping.*
- *1 mark for the hexadecimal result.*

b) Answer: 0xC1360000 = -11.37510

- *1 mark for the sign bit.*
- *1 mark for the exponent field.*
- *1 mark for the fraction field.*
- *1 mark for applying the bias and significand.*
- *1 mark for the final decimal value.*

© Hak Cipta Universiti Teknologi MARA

PART B — MARKING GUIDE

QUESTION B1 (20 marks)

Topic: Assembly-language programming

Source: CSC429 (18).pdf, page(s) 6

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

Assumption: Figure 2 is interpreted as y = A - ((2C * D) / (B + A^2)). The expression is evaluated as: temp1 = 2*C; temp2 = temp1*D; temp3 = A*A; temp4 = B + temp3; temp5 = temp2 / temp4; y = A - temp5. Registers A, B, C, D are fixed; x, y, z are temporary. Final result must be in y.

One-address program (accumulator-based):

```
LOAD A       ; ACC = A
MUL A        ; ACC = A^2
ADD B        ; ACC = A^2 + B
STOR z       ; z = A^2 + B
LOAD C       ; ACC = C
MUL D        ; ACC = C*D
ADD C        ; ACC = 2*C*D  (since C*D + C*D = 2*C*D)
DIV z        ; ACC = (2*C*D)/(A^2+B)
STOR x       ; x = (2*C*D)/(A^2+B)
LOAD A       ; ACC = A
SUB x        ; ACC = A - x
STOR y       ; y = final answer
```

Two-address program (operand1 = operand1 op operand2):

```
MOV x, C     ; x = C
ADD x, C     ; x = 2C
MUL x, D     ; x = 2C*D
MOV z, A     ; z = A
MUL z, A     ; z = A^2
ADD z, B     ; z = B + A^2
DIV x, z     ; x = (2C*D)/(B + A^2)
MOV y, A     ; y = A
SUB y, x     ; y = A - x
```

Three-address program (dest = src1 op src2):

```
MUL x, C, D   ; x = C*D
ADD x, x, x   ; x = 2*C*D
MUL z, A, A   ; z = A^2
ADD z, B, z   ; z = B + A^2
DIV x, x, z   ; x = (2*C*D)/(B + A^2)
SUB y, A, x   ; y = A - x
```

Unofficial marking guide:

- *1 mark: correct interpretation of Figure 2 as y = A - ((2C*D)/(B + A^2)) and correct order of operations.*
- *6 marks: one-address program – 1 mark for correct use of LOAD/STOR, 1 mark for computing A^2+B, 1 mark for computing 2C*D, 1 mark for division, 1 mark for final subtraction, 1 mark for storing final answer in y.*
- *6 marks: two-address program – 1 mark for MOV to initialise, 1 mark for computing 2C, 1 mark for 2C*D, 1 mark for A^2+B, 1 mark for division, 1 mark for final subtraction into y.*
- *6 marks: three-address program – 1 mark for MUL x,C,D, 1 mark for doubling to 2C*D, 1 mark for MUL z,A,A, 1 mark for ADD z,B,z, 1 mark for DIV x,x,z, 1 mark for SUB y,A,x.*
- *1 mark: correct assembly notation (mnemonics and operand order) consistent with Table 1 for all three programs.*

Model confidence: medium.

*Model note: Figure 2 is not fully legible from the extracted text; the expression is assumed to be y = A - (2C*D)/(B + A^2). If the figure shows a different grouping (e.g., denominator B + A^2 or (B+A)^2), the programs must be adjusted accordingly. Table 1 uses inconsistent punctuation (e.g., ADD (p.q.r)); operand order is taken as dest, src1, src2 for three-address and dest, src for two-address. One-address instructions are assumed to use an implicit accumulator.*

END OF MARKING SCHEME

© Hak Cipta Universiti Teknologi MARA